

ARK-457 — Cross-Context Authorization Replay (Confused Deputy)

PREREGISTRATION (committed before execution)

Experiment ID: ARK-457

Series: ExecutionProof authorization-boundary corpus (enterprise-failure-mode phase)

Substrate: Classical software (no quantum hardware, no cryptography)

Question: When an authorization is issued bound to a specific context (tenant, session, resource, audience, environment), can it be replayed to authorize an execution under a **different** context? A correct guard must ALLOW execution only when the execution context matches the authorization's bound context **exactly** on every binding dimension, and DENY otherwise — including superficial “confusable” near-matches.

1. Motivation

The **confused deputy** problem is one of the oldest and most damaging authorization failure modes: a privileged component is tricked into misusing an authorization that was legitimately issued for a different context. In modern agent and enterprise systems this shows up as:

- **Cross-tenant replay** — an authorization minted for tenant A is presented against tenant B's resources (multi-tenant isolation break).
- **Cross-session replay** — a token/authorization from one session is reused in another (session fixation / token export).
- **Cross-resource replay** — approval for wallet/resource X is redirected to resource Y.
- **Cross-audience replay** — an authorization scoped to `api.reporting` is presented to `api.payments` (audience confusion / scope creep).
- **Cross-environment replay** — a `staging / sandbox` authorization is replayed against `production`.

The common defect is a guard that checks that an authorization exists and is valid, but not that the execution context is the one the authorization was bound to. A frequent aggravating factor is **lenient comparison** — a comparator that normalizes case, trims whitespace, ignores zero-width characters, or maps homographs, thereby treating a genuinely different context string as “the same.”

This experiment tests: Does the guard bind an authorization to its issuing context and refuse to let it transfer to any other context — using strict, exact comparison that is not fooled by confusable near-matches?

2. Design

Structure

- **8 arms** × 100 scenarios per arm = **800 evaluation decisions**
- Each arm represents a specific context relationship between authorization and execution

- Dual independent guards (V1 JavaScript, V2 Python) evaluate each scenario

Context Tuple (Binding Dimensions)

Every authorization is bound to a 5-dimension context tuple. Execution presents its own tuple.

Dimension	Role	Example values
tenant	Isolation boundary / customer	tenant-0007
session	Session / token scope	sess-0002af
resource	Governed object	wallet-0031
audience	Intended API/service scope	api.payments , api.reporting
environment	Deployment stage	production , staging , sand- box , dr-failover

Authorization + Execution

```
{
  "authorization": {
    "action": "transfer" || "approve" || "delegate" || "execute",
    "subject": "agent-X",
    "binding": { "tenant": ..., "session": ..., "resource": ..., "audience": ..., "environment": ... }
  },
  "execution": {
    "context": { "tenant": ..., "session": ..., "resource": ..., "audience": ..., "environment": ... }
  }
}
```

Decision Procedure (Both Guards)

```
evaluate(authorization, execution):
  for dim in [tenant, session, resource, audience, environment]:
    a = authorization.binding[dim]
    b = execution.context[dim]
    if a and b are not both strings, OR a != b  # byte/code-point EXACT equality
      → return DENY (authorization does not transfer across contexts)
  → return ALLOW (execution context matches bound context on every dimension)
```

No normalization, no case folding, no whitespace trimming, no zero-width stripping, no homoglyph mapping. A confusable near-match is a **MISMATCH** → **DENY**.

3. Arms

Arm	Condition	Mismatch	Expected
1	BASELINE-exact-context-match	none (all 5 dims identical)	ALLOW
2	CROSS-TENANT	tenant differs	DENY
3	CROSS-SESSION	session differs	DENY
4	CROSS-RESOURCE	resource differs	DENY
5	CROSS-AUDIENCE	audience differs	DENY
6	CROSS-ENVIRONMENT	environment differs	DENY
7	MULTI-DIMENSION-mismatch	≥ 2 dims differ	DENY
8	CONFUSABLE-near-match	exactly 1 dim differs by a superficial variant (trailing space, case flip, zero-width char, or homoglyph) that a lenient comparator would wrongly merge	DENY

- **Arm 1** (baseline): exact match on all five dimensions → expected **ALLOW**
- **Arms 2-8**: at least one binding dimension does not match under exact equality → expected **DENY** (authorization must not replay across contexts)
- **Arm 8** specifically tests that the guard is not fooled by confusable strings: the differing value is byte-different but would collapse to the original under naive normalization.

4. Pass Criteria (Preregistered)

Let:

- **g_allow_exact** = ALLOW rate in Arm 1 (exact context match)
- **g_deny_crosscontext_min** = minimum DENY rate across Arms 2-8 (cross-context replay)
- **g_margin** = $\min(\text{g_allow_exact}, \text{g_deny_crosscontext_min}) - 0.05$

Criteria:

- **C1:** $\text{g_allow_exact} \geq 0.95$
(Legitimate same-context executions proceed)
- **C2:** $\text{g_deny_crosscontext_min} \geq 0.95$
(Every cross-context replay class is refused — including confusable near-matches)

- **C3:** $g_margin \geq 0.90$

(Strong separation between exact-allow and cross-context-deny)

Verdict: PASS if $C1 \wedge C2 \wedge C3$, else FAIL. Recorded verdict stands as executed.

5. Context-Replay Effectiveness Gate

Before evaluating any arm, a **structural oracle** (`context_replay_effective`) verifies that each scenario genuinely encodes its intended context relationship under **exact equality**:

- Arm 1: **0** dimensions differ.
- Arms 2–6: **exactly one** specified dimension differs, all others identical.
- Arm 7: ≥ 2 dimensions differ.
- Arm 8: **exactly one** dimension differs by a genuine byte difference **that a naive normalizer would merge** (confirmed confusable).

If any scenario in an arm fails the effectiveness check, the arm is **aborted** — the experiment does not proceed with inert test cases. This gate prevents ARK-455-style no-op defects where a scenario appears to test a condition but does not actually encode it.

Abort condition: If any scenario in any arm is not context-replay-effective, the arm run is aborted and the experiment is stopped.

6. Dual Independent Guards

V1 Guard (JavaScript, no dependencies)

- Pure logic, no external libraries
- Compares `execution.context` against `authorization.binding` with strict `!==` equality per dimension
- Outputs: `{decision: "ALLOW" | "DENY", reason: "..."}`

V2 Guard (Python)

- Independent implementation of the same procedure
- Compares with strict `!=` equality per dimension
- Outputs: `{decision: "ALLOW" | "DENY", reason: "..."}`

Cross-runtime note: All scenario values remain within the Unicode BMP so that JavaScript (UTF-16) and Python (code-point) exact-equality comparisons are identical. This ensures concordance reflects the decision logic, not encoding artifacts.

Concordance requirement: V1 and V2 must agree on every decision. Disagreement indicates ambiguity in the procedure definition.

7. Kill-Gate Calibration

Before running the 8 arms, generate 100 calibration scenarios (mixed across all 8 arms) and evaluate with both guards. Verify:

1. **Context-replay effectiveness:** All 100 scenarios pass the structural oracle
2. **V1-V2 concordance:** $\geq 99\%$ agreement

If the kill-gate fails, the experiment is **aborted** — the dual-guard setup is not ready.

8. Execution Procedure

1. **LOCK** — Commit this preregistration, schema, dual guards, generator, runners, and a SHA-256 MANIFEST **before** any scenario is generated or evaluated
2. **Kill-gate** — Generate 100 calibration scenarios; check effectiveness and concordance
3. **Arm execution** — For each arm 1-8:
 - Generate 100 scenarios (deterministic seed = SEED_BASE + arm)
 - Verify context-replay effectiveness (abort if any scenario fails)
 - Evaluate with V1 guard
 - Evaluate with V2 guard
 - Record concordance
4. **Compute overall metrics** — g_allow_exact, g_deny_crosscontext_min, g_margin
5. **Apply criteria** — C1, C2, C3 → VERDICT
6. **Record results** — Fill MANIFEST OUTCOME; commit results; create tags

9. Constraints and Scope

What this tests:

- Whether a guard binds an authorization to its issuing context and refuses to transfer it to any other context
- Whether the comparison is strict enough to reject confusable near-matches
- Whether the decision procedure is clear enough for independent implementation (dual-guard concordance)

What this does NOT test:

- Real token formats, JWT/PASETO audience claims, or OAuth scope semantics
- Cryptographic binding (signatures, key confirmation, DPoP) — this is logical context binding only
- Network-level replay (TLS, nonce caches) or timing/expiry (covered by ARK-442/ARK-451)
- Unicode normalization policy design — here, strict non-normalization is the tested behavior by construction
- Performance, latency, or throughput under load

Constraints: This is a classical/software boundary-logic testbed validating context-binding in isolation, not the end-to-end enterprise identity stack. Findings are scoped to the tested procedure, scenario generator, and arms.

10. Provenance and Integrity

- **LOCK before execution:** Preregistration + schema + dual guards + generator + runners + MANIFEST (SHA-256) committed before any scenario generated or evaluated

- **No post-hoc criterion changes:** The three pass criteria (C1, C2, C3) are fixed at LOCK time
 - **Honest reporting:** If the guard binds context correctly, report PASS. If any cross-context replay succeeds (ALLOW), report honest FAIL.
 - **Commits not cryptographically signed:** Provenance = commit history + MANIFEST SHA-256 hashes
-

11. Real-World Relevance

Confused-deputy / cross-context replay underlies many high-impact incidents:

- Multi-tenant SaaS isolation breaks (authorization for tenant A accepted for tenant B)
- OAuth/token audience confusion (token for one service accepted by another)
- Session token export/replay across sessions or devices
- Staging/sandbox credentials accepted in production
- Homoglyph/whitespace tricks defeating naive identifier comparison

A guard that validates an authorization without re-binding it to the exact execution context is exploitable. ARK-457 validates strict, non-normalizing context binding.

12. Expected Outcome

If the guard binds context correctly:

- **Arm 1 (exact match):** ALLOW rate $\approx$ 100%
- **Arms 2-8 (cross-context, incl. confusable):** DENY rate $\approx$ 100%
- **V1-V2 concordance:** 100%
- **VERDICT:** PASS

If the guard leaks authorization across contexts (e.g. via lenient comparison):

- **Some of Arms 2-8:** ALLOW rate $>$ 0% (dangerous — authorization replayed across contexts)
 - **VERDICT:** FAIL (honest)
-

Investigator: Derek Hone, Remnant Fieldworks Inc.

Executor: Abacus.AI autonomous agent (supervised)

Series: ExecutionProof™ authorization-boundary corpus

Trademarks: ExecutionProof™, ProofRecord™, VaultProof™, Verification Before Execution™, Proof Before Power™

If it cannot be verified, it cannot execute.